collators.py

`src/moe_pruning/data/collators.py · 92 lines · module moe_pruning.data.collators`

What it does

Turns lists of tokenized examples into padded tensor batches (`ModelBatch`) that also carry the real-token mask of each model stack (`encoder`, `decoder`).

How it works

- All sequences are right-padded. For Switch Transformers this matters because the router assigns expert capacity in sequence order, so right padding can never steal capacity from real tokens.
- `collate_seq2seq()`: encoder mask from the attention mask; decoder mask from `labels != -100` (decoder position j receives label j-1, so these are exactly the real decoder positions).
- `collate_causal()`: one `decoder` mask equal to the attention mask; labels decide which tokens are scored.
- `ModelBatch.model_inputs()` moves tensors to the input device; `token_masks()` returns the per-stack masks.

Notes

- Padding is excluded from every statistic and loss through these masks.

Defined in this file

Extracted from the source code (classes, public functions and methods).

Name	Description
`class ModelBatch`	Padded batch with per-stack real-token masks.
`.model_inputs(device)`	input_ids, attention_mask and labels moved to a device.
`.token_masks(device)`	Real-token mask of each stack (encoder / decoder).
`collate_seq2seq(items, pad_token_id)`	Encoder-decoder batch.
`collate_causal(items, pad_token_id)`	Decoder-only batch: `labels` has the same length as `input_ids`.
`iter_batches(items, batch_size, pad_token_id)`	Encoder-decoder batches (kept for compatibility; prefer `task.iter_batches`).

Connections

Used by: `src/moe_pruning/tasks/base.py`, `src/moe_pruning/tasks/causal_lm.py`, `src/moe_pruning/tasks/seq2seq_denoising.py`